# Chapter 14 | Exercise solutions with Python

*Selective State Spaces*

This notebook solves Exercises 5–9. It checks tiny zero-order-hold steps, compares coordinate-specific and shared retentions, and traces the same selective update through dense, recurrent, and chunked calculations. It then tests causality and the effect of a zero retention inside a chunk. Exercises 1–4 and 10 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch14_selective_ssm"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

## 5. Inspect a small step numerically

Hold the input fixed during a short interval of length `delta`. In the chapter's scalar zero-order-hold example, the fraction of the old state that survives the interval is $a = e^{-\Delta}$. The coefficient multiplying the input is $b = 1 - e^{-\Delta}$. If the interval is very short, the state barely moves and $b$ is small. Its size is about $\Delta$, since $e^{-\Delta} = 1 - \Delta + \Delta^2/2 - \cdots$ near zero.

Computing `1 - np.exp(-delta)` first rounds the exponential to a floating-point value near one. The subtraction exposes the small difference, so an error of order $10^{-16}$ in that rounded value can be a large fraction of the write. This loss of significant digits is called **cancellation**. The helper instead uses `np.expm1(-delta)`. The `expm1` operation evaluates $e^z - 1$ directly and is designed to retain accuracy when $z$ is close to zero.

```
from decimal import Decimal, localcontext

from examples.ch14_selective_ssm.model import zoh_coefficients

delta = np.array([1e-4, 1e-9, 1e-16], dtype=np.float64)
retain, write = zoh_coefficients(delta)
subtract = 1.0 - np.exp(-delta)

# Compute the reference at 80 decimal digits, then round once.
with localcontext() as context:
    context.prec = 80
    reference = np.array([
        float(1 - (-Decimal.from_float(float(step))).exp())
        for step in delta
    ])

np.testing.assert_allclose(write, reference, rtol=2e-15, atol=0)
assert np.all(write > 0)
print(f"{'delta':>8}  {'stable write':>23}  {'subtraction':>23}")
for step, stable, direct in zip(delta, write, subtract):
    print(f"{step:8.1e}  {stable:23.16e}  {direct:23.16e}")
```

```
OUTPUT
   delta              stable write              subtraction
 1.0e-04   9.9950000166662508e-05   9.9950000166663850e-05
 1.0e-09   9.9999999949999999e-10   9.9999997171806854e-10
 1.0e-16   9.9999999999999998e-17   1.1102230246251565e-16
```

The reference uses the same binary float64 input values as NumPy. It performs the exponential and subtraction at much higher precision before rounding the small result to float64. The check allows a relative tolerance of $2 \times 10^{-15}$ and no absolute tolerance, so an erroneous zero would fail even for the smallest positive write.

To measure the loss of precision, divide the absolute error by the reference write for each step. Also try one extra, smaller step to see when subtraction can return zero.

```
relative_stable = np.abs(write - reference) / reference
relative_subtract = np.abs(subtract - reference) / reference

print(f"{'delta':>8}  {'stable relative err':>19}  "
      f"{'subtract relative err':>21}")
for step, good, bad in zip(delta, relative_stable, relative_subtract):
    print(f"{step:8.1e}  {good:19.3e}  {bad:21.3e}")

tiny_delta = np.float64(1e-17)
tiny_retain, tiny_write = zoh_coefficients(tiny_delta)
print(f"Extra step: delta={tiny_delta:.1e}, retain={tiny_retain:.17g}")
print(f"1 - retain = {1.0 - tiny_retain:.17g}")
print(f"Stable write = {tiny_write:.17g}")
```

```
OUTPUT
   delta  stable relative err  subtract relative err
 1.0e-04            0.000e+00              1.342e-14
 1.0e-09            0.000e+00              2.778e-08
 1.0e-16            0.000e+00              1.102e-01
Extra step: delta=1.0e-17, retain=1
1 - retain = 0
Stable write = 1.0000000000000001e-17
```

In this run, at $10^{-4}$ the two writes differ in their last few digits. At $10^{-9}$ the direct subtraction has a relative error of about $2.8 \times 10^{-8}$. At $10^{-16}$, the direct subtraction returns about $1.11022 \times 10^{-16}$, the gap between one and the float64 value immediately below it. The stable result is about $10^{-16}$, so the subtraction is about 11 percent too large despite its small absolute error.

The exact last digits can vary with the implementation of the exponential. Whenever its rounded value is exactly one, subtraction produces zero. The extra step $\Delta = 10^{-17}$ shows this case. The helper preserves the small input coefficient even though the returned retention rounds to one. The mathematical coefficients still sum to one before rounding.

## 6. Give the coordinates different retentions

Now let the state hold two numbers. A row of `a` supplies the retention for each coordinate at that time step. The rows of `b` are all $(1, 1)$, so each new scalar input is written to both coordinates without scaling. The rows of `c` are also $(1, 1)$, so the readout is the sum of the two coordinates after the update.

With separate retentions, the update is

$$x_{t,1} = \tfrac{1}{4} x_{t-1,1} + u_t, \qquad x_{t,2} = \tfrac{3}{4} x_{t-1,2} + u_t, \qquad y_t = x_{t,1} + x_{t,2}.$$

These write coefficients are specified independently of the retention. The rule $b = 1 - a$ in Exercise 5 came from one particular scalar system.

| Time $t$ | Input $u_t$ | State $(x_{t,1}, x_{t,2})$ | Readout |
|---|---|---|---|
| 1 | 2 | $(0 + 2,\ 0 + 2) = (2, 2)$ | 4 |
| 2 | 0 | $(2/4,\ 3(2)/4) = (1/2, 3/2)$ | 2 |
| 3 | 1 | $(1/8 + 1,\ 9/8 + 1) = (9/8, 17/8)$ | 13/4 |

The second input is zero, which isolates the two retentions. The coordinates enter that step with the same value, 2. The first carries $1/2$ forward and the second carries $3/2$ forward because it retains a larger fraction.

```
from examples.ch14_selective_ssm.model import selective_scan

scan_u = np.array([2.0, 0.0, 1.0])
scan_a = np.tile([0.25, 0.75], (len(scan_u), 1))
scan_b = np.ones((len(scan_u), 2))
scan_c = np.ones_like(scan_b)
assert scan_a.shape == scan_b.shape == scan_c.shape == (3, 2)
separate_y, separate_states = selective_scan(
    scan_u, scan_a, scan_b, scan_c,
)

expected_separate = np.array([[2, 2], [1/2, 3/2], [9/8, 17/8]])
np.testing.assert_allclose(
    separate_states, expected_separate, rtol=0, atol=1e-12,
)
np.testing.assert_allclose(separate_y, [4, 2, 13/4], rtol=0,
                           atol=1e-12)
print("Separate retention shape", scan_a.shape)
print("States (one row per input)")
print(separate_states)
print("Outputs", separate_y)
```

```
OUTPUT
Separate retention shape (3, 2)
States (one row per input)
[[2.    2.   ]
 [0.5   1.5  ]
 [1.125 2.125]]
Outputs [4.   2.   3.25]
```

Give a shape $[3, 1]$ to store one retention per time step. The scan multiplies a length-one row by the two-coordinate state. NumPy **broadcasts** the length-one row, meaning its single value is used for both multiplications. The state, write, and read widths remain two.

Starting from equal states, the coordinates now receive the same retention and the same write, so they stay equal. After reading 2, each coordinate is 2. After reading 0, each is $2/4 = 1/2$. The final input of 1 gives $(1/4)(1/2) + 1 = 9/8$ per coordinate. The outputs are therefore $(4, 1, 9/4)$. Use absolute tolerance $10^{-12}$ and zero relative tolerance for these float64 state and output checks.

```
shared_a = np.full((len(scan_u), 1), 0.25)
shared_y, shared_states = selective_scan(
    scan_u, shared_a, scan_b, scan_c,
)
expected_shared = np.array([[2, 2], [1/2, 1/2], [9/8, 9/8]])
np.testing.assert_allclose(
    shared_states, expected_shared, rtol=0, atol=1e-12,
)
np.testing.assert_allclose(
    shared_y, [4, 1, 9/4], rtol=0, atol=1e-12,
)
np.testing.assert_allclose(
    separate_y - shared_y, [0, 1, 1], rtol=0, atol=1e-12,
)

print("Shared retention shape", shared_a.shape)
print("States (one row per input)")
print(shared_states)
print("Outputs", shared_y)
print("Separate minus shared", separate_y - shared_y)
```

OUTPUT

```
Shared retention shape (3, 1)
States (one row per input)
[[2.    2.   ]
 [0.5   0.5  ]
 [1.125 1.125]]
Outputs [4.   1.   2.25]
Separate minus shared [0. 1. 1.]
```

The first outputs agree because the initial state is zero and both runs apply the same write. The last two outputs in the separate-retention run are each one larger. The first coordinate uses $1/4$ in both runs and contributes no difference. At time $2$ the second coordinate differs by $3/2 - 1/2 = 1$. At time $3$ its retained contributions differ by $(3/4)(3/2) - (1/4)(1/2) = 1$ before the same new input is added to both. This is a difference in the stored state that the readout exposes.

## 7. Move the chunk boundary

A chunk is a group of consecutive positions processed together. Changing its size moves the boundary where the algorithm must pass its state forward. The sequence and the recurrence are the same for every boundary choice.

Here each state has one coordinate, and each write and read coefficient is one. Starting from $x_0 = 0$, the recurrence is $x_t = a_t x_{t-1} + u_t$ and the output is $y_t = x_t$. The manuscript numbers positions from one. NumPy row $0$ therefore refers to position $1$ in the calculations below.

| Position $t$ | Retention $a_t$ | Input $u_t$ | Retained state | State and output $x_t = y_t$ |
|---|---|---|---|---|
| 1 | 1/2 | 1 | 0 | 1 |
| 2 | 1/4 | 2 | 1/4 | 9/4 |
| 3 | 3/4 | 0 | 27/16 | 27/16 |
| 4 | 1/2 | 3 | 27/32 | 123/32 |
| 5 | 1/4 | 1 | 123/128 | 251/128 |

In the dense calculation, row $t$ of $M$ tells us how each input contributes to $y_t$. An input written at $s$ enters with coefficient one. It passes through the retentions at positions $s+1$ through $t$ before it is read. Thus $M_{t,s} = \prod_{i=s+1}^{t} a_i$ for $s \le t$. The empty product on the diagonal equals one. For $s > t$ the input is still in the future and its coefficient is zero.

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1/4 & 1 & 0 & 0 & 0 \\ 3/16 & 3/4 & 1 & 0 & 0 \\ 3/32 & 3/8 & 1/2 & 1 & 0 \\ 3/128 & 3/32 & 1/8 & 1/4 & 1 \end{bmatrix}.$$

For instance, the input at position 2 passes through $a_3 = 3/4$ and $a_4 = 1/2$ to reach output 4. Its coefficient is $3/8$, which multiplies $u_2 = 2$. Multiplying the whole fourth row by $u$ gives $3/32 + (3/8)2 + (1/2)0 + 3 = 123/32$, exactly the fourth output of the recurrence.

```
from examples.ch14_selective_ssm.model import (
    ssd_chunked,
    ssd_dense,
    ssd_recurrent,
)

chunk_u = np.array([1.0, 2.0, 0.0, 3.0, 1.0])
chunk_a = np.array([0.5, 0.25, 0.75, 0.5, 0.25])
chunk_b = np.ones((5, 1))
chunk_c = np.ones((5, 1))
chunk_atol = 1e-12

chunk_dense, chunk_matrix = ssd_dense(
    chunk_u, chunk_a, chunk_b, chunk_c
)
chunk_recurrent, chunk_states = ssd_recurrent(
    chunk_u, chunk_a, chunk_b, chunk_c
)
```

Check the matrix and both output vectors against the hand calculations.

```
expected_matrix = np.array([
    [1, 0, 0, 0, 0],
    [1/4, 1, 0, 0, 0],
    [3/16, 3/4, 1, 0, 0],
    [3/32, 3/8, 1/2, 1, 0],
    [3/128, 3/32, 1/8, 1/4, 1],
])
expected_output = np.array([1, 9/4, 27/16, 123/32, 251/128])
np.testing.assert_allclose(
    chunk_matrix, expected_matrix, rtol=0, atol=chunk_atol
)
np.testing.assert_allclose(
    chunk_dense, expected_output, rtol=0, atol=chunk_atol
)
np.testing.assert_allclose(
    chunk_recurrent, expected_output, rtol=0, atol=chunk_atol
)
np.testing.assert_allclose(
    chunk_states[:, 0], expected_output, rtol=0, atol=chunk_atol
)

print("Dense sequence matrix")
print(chunk_matrix)
print("Dense outputs    ", chunk_dense)
print("Recurrent outputs", chunk_recurrent)
print("State trace      ", chunk_states[:, 0])
```

```
OUTPUT
Dense sequence matrix
[[1.        0.        0.        0.        0.       ]
 [0.25      1.        0.        0.        0.       ]
 [0.1875    0.75      1.        0.        0.       ]
 [0.09375   0.375     0.5       1.        0.       ]
 [0.0234375 0.09375   0.125     0.25      1.       ]]
Dense outputs     [1.        2.25      1.6875    3.84375   1.9609375]
Recurrent outputs [1.        2.25      1.6875    3.84375   1.9609375]
State trace       [1.        2.25      1.6875    3.84375   1.9609375]
```

A chunk first computes what its own inputs would produce with a zero incoming state. Then it adds the contribution of the actual incoming state, which is multiplied by every retention from the start of the chunk through the position being read. Those are two different products. A local write skips its own position's retention, while the incoming state has to pass through that retention.

With chunk size 3, the first group covers positions 1–3 and leaves state $x_3 = 27/16$. The group at positions 4–5 is shorter. Its inputs $(3, 1)$ would give $(3, 7/4)$ if this group started from zero. The state from position 3 adds $(27/32, 27/128)$, because it is first multiplied by $1/2$ and then by $1/4$. The sum is $(123/32, 251/128)$. The final state is $251/128$.

Chunk size 1 passes a state after every position. Size 5 contains the whole sequence in one block. Sizes 2 and 3 both need a shorter last block. The assertions below use float64, with an absolute tolerance of $10^{-12}$ and zero relative tolerance. Compare each output array to both references and the returned state to the final recurrent row of shape `[1]`.

```
print("Size  Block lengths     Max |error|   Final state")
for size in (1, 2, 3, 5):
    output, final = ssd_chunked(
        chunk_u, chunk_a, chunk_b, chunk_c, chunk_size=size
    )
    for reference in (chunk_dense, chunk_recurrent):
        np.testing.assert_allclose(
            output, reference, rtol=0, atol=chunk_atol
        )
    np.testing.assert_allclose(
        final, chunk_states[-1], rtol=0, atol=chunk_atol
    )
    lengths = [
        min(size, len(chunk_u) - start)
        for start in range(0, len(chunk_u), size)
    ]
    error = np.max(np.abs(output - chunk_recurrent))
    print(f"{size:>4}  {str(lengths):<17} {error:.2e}     {final[0]}")
```

```
OUTPUT
Size  Block lengths     Max |error|   Final state
   1  [1, 1, 1, 1, 1]   0.00e+00     1.9609375
   2  [2, 2, 1]         0.00e+00     1.9609375
   3  [3, 2]            0.00e+00     1.9609375
   5  [5]               0.00e+00     1.9609375
```

All four chunk sizes agree with the hand calculation. The full sequence matrix displays every possible input–output path in one array. The chunked method evaluates only local blocks directly. It carries the effect of previous blocks in the state at each boundary, which is sufficient for this recurrence.

## 8. Check causality within a chunk (optional)

Keep the five-position example from Exercise 7 and use chunks of sizes 3 and 2. Change only the third input, from 0 to 4. The first two inputs now share a chunk with an edited future input. We want their outputs to remain unchanged even though the code computes the chunk together.

The local matrix of the first chunk is the upper-left $3 \times 3$ block of $M$.

$$M_{local} = \begin{bmatrix} 1 & 0 & 0 \\ 1/4 & 1 & 0 \\ 3/16 & 3/4 & 1 \end{bmatrix}.$$

The first two rows have zeros in column 3. The third row has a one there because the current input is written before its state is read. The change of 4 therefore adds $(0, 0, 4)$ to the first chunk's outputs. The change to its outgoing state is also 4. It passes through retentions $1/2$ and $1/4$ in the next chunk, adding 2 at position 4 and $1/2$ at position 5. The output difference across the full sequence should be $(0, 0, 4, 2, 1/2)$.

```
future_u = chunk_u.copy()
future_u[2] = 4.0
before_edit, _ = ssd_chunked(
    chunk_u, chunk_a, chunk_b, chunk_c, chunk_size=3
)
after_edit, _ = ssd_chunked(
    future_u, chunk_a, chunk_b, chunk_c, chunk_size=3
)
edit_difference = after_edit - before_edit

np.testing.assert_array_equal(before_edit[:2], after_edit[:2])
np.testing.assert_allclose(
    edit_difference, [0, 0, 4, 2, 0.5], rtol=0, atol=chunk_atol
)
np.testing.assert_allclose(
    edit_difference, 4 * chunk_matrix[:, 2], rtol=0, atol=chunk_atol
)

print("First chunk's local matrix")
print(chunk_matrix[:3, :3])
print("Before", before_edit)
print("After ", after_edit)
print("Change", edit_difference)
```

OUTPUT

```
First chunk's local matrix
[[1.     0.     0.    ]
 [0.25   1.     0.    ]
 [0.1875 0.75   1.    ]]
Before [1.        2.25      1.6875    3.84375   1.9609375]
After  [1.        2.25      5.6875    5.84375   2.4609375]
Change [0.  0.  4.  2.  0.5]
```

The first two outputs remain $(1, 2.25)$. Output 3 increases by 4 and output 4 increases by 2. The triangular local matrix prevents the edit from moving backward within its own chunk, while the state at the boundary carries the edit forward to the next one.

## 9. Reset a state inside a chunk (optional)

Use chunk size 3 again. Set $u_3 = 4$ and this time also set $a_3 = 0$. Compare a warm start with $x_0 = 2$ to a cold start with $x_0 = 0$. Since $b_t = c_t = 1$ for every position, the state is again the output. The first two transitions still let the initial state contribute. At position 3, the update is $x_3 = 0 \cdot x_2 + 1 \cdot 4 = 4$ in both runs.

| Position $t$ | Warm output | Cold output | Warm minus cold |
|---|---|---|---|
| 1 | $(1/2)2 + 1 = 2$ | 1 | 1 |
| 2 | $(1/4)2 + 2 = 5/2$ | 9/4 | 1/4 |
| 3 | 4 | 4 | 0 |
| 4 | $(1/2)4 + 3 = 5$ | 5 | 0 |
| 5 | $(1/4)5 + 1 = 9/4$ | 9/4 | 0 |

The two runs have the same inputs and coefficients. Their state difference starts at 2 and evolves by $\delta_t = a_t \delta_{t-1}$. Thus the contribution of the initial state to output $t$ is $2 \prod_{i=1}^{t} a_i$. The product contains a zero starting at position 3. This predicts that only outputs 1 and 2 depend on which initial state we choose.

```
reset_u = chunk_u.copy()
reset_a = chunk_a.copy()
reset_u[2] = 4.0
reset_a[2] = 0.0

warm, warm_final = ssd_chunked(
    reset_u, reset_a, chunk_b, chunk_c, chunk_size=3, x0=[2.0]
)
cold, cold_final = ssd_chunked(
    reset_u, reset_a, chunk_b, chunk_c, chunk_size=3, x0=[0.0]
)
```

Check the warm and cold runs against the references separately.

```
for x0, output, final, expected in (
    ([2.0], warm, warm_final, [2, 2.5, 4, 5, 2.25]),
    ([0.0], cold, cold_final, [1, 2.25, 4, 5, 2.25]),
):
    dense, reset_matrix = ssd_dense(
        reset_u, reset_a, chunk_b, chunk_c, x0=x0
    )
    recurrent, states = ssd_recurrent(
        reset_u, reset_a, chunk_b, chunk_c, x0=x0
    )
    for reference in (dense, recurrent, expected):
        np.testing.assert_allclose(
            output, reference, rtol=0, atol=chunk_atol
        )
    np.testing.assert_allclose(
        final, states[-1], rtol=0, atol=chunk_atol
    )

print("Warm start", warm)
print("Cold start", cold)
print("Difference", warm - cold)
print("Final state, warm and cold", warm_final, cold_final)
```

```
OUTPUT
Warm start [2.   2.5  4.   5.   2.25]
Cold start [1.   2.25 4.   5.   2.25]
Difference [1.   0.25 0.   0.   0.  ]
Final state, warm and cold [2.25] [2.25]
```

The reset acts on the old state. The new write $b_3 u_3$ is added after that multiplication, so it can still affect position 3 and later outputs. In the matrix, an earlier write must cross $a_3$ to reach position 3 or beyond, so those coefficients become zero. The coefficient for the current input is $M_{3,3} = 1$. The row of the sequence matrix at position 3 is consequently $(0, 0, 1, 0, 0)$. The first chunk now passes state 4 onward, regardless of which initial state entered it.

```
initial_response = 2 * np.cumprod(reset_a)
np.testing.assert_allclose(
    warm - cold, initial_response, rtol=0, atol=chunk_atol
)
np.testing.assert_array_equal(reset_matrix[2], [0, 0, 1, 0, 0])
np.testing.assert_array_equal(reset_matrix[2:, :2], np.zeros((3, 2)))
print("Initial-state response", initial_response)
print("Matrix row at position 3", reset_matrix[2])
```

```
OUTPUT
Initial-state response [1.   0.25 0.   0.   0.  ]
Matrix row at position 3 [0. 0. 1. 0. 0.]
```